

Answer Key

Exercise 1: c) 24

Exercise 2: c) 15

Exercise 3: See solution below

Exercise 4: See solution below

Exercise 5: See solution below

Exercise 6: See solution below

Exercise 7: See solution below

Exercise 8: See solution below

Exercise 9: b) ACB

Exercise 10: b) DogCatAnimal

Exercise 11: See solution below

Exercise 12: See solution below

Exercise 13: See solution below

Exercise 14: See solution below

Exercise 15: b) To reduce the number of system calls by reading/writing data in chunks

Exercise 16: a) The file will not be closed if an exception occurs

Solutions for Code Exercises

Exercise 3 - Solution

```
static int sumOfDigits(int n) {  
    if (n == 0) {  
        return 0;  
    } else {  
        return (n % 10) + sumOfDigits(n / 10);  
    }  
}
```

Exercise 4 - Solution

```
ArrayList<String> allBinaryStrings(int n) {  
    ArrayList<String> result = new ArrayList<String>();  
    if (n == 0) {  
        result.add("");  
    } else {  
        ArrayList<String> rest = allBinaryStrings(n - 1);  
        for (String s : rest) {  
            result.add("0" + s);  
            result.add("1" + s);  
        }  
    }  
}
```

```
    return result;
}
```

Exercise 5 - Solution

```
static void removeLast(Node p) {
    // Find the first node
    while (p.prev != null) {
        p = p.prev;
    }
    // If list is empty
    if (p == null) return;
    // If only one node
    if (p.next == null) return;
    // Traverse to last node
    Node current = p;
    while (current.next != null) {
        current = current.next;
    }
    // Remove last node
    if (current.prev != null) {
        current.prev.next = null;
    }
}
```

Exercise 6 - Solution

```
void addAtBeginning(int x) {
    Node newNode = new Node();
    newNode.element = x;
    newNode.next = this.first;
    this.first = newNode;
}
```

Exercise 7 - Solution

```
static <T> int countNodes(TreeNode<T> root) {
    if (root == null) {
        return 0;
    } else {
        return 1 + countNodes(root.leftChild) +
countNodes(root.rightChild);
    }
}
```

Exercise 8 - Solution

```
static <T> void mirrorTree(TreeNode<T> root) {
    if (root == null) {
        return;
    }
    // Swap children
    TreeNode<T> temp = root.leftChild;
    root.leftChild = root.rightChild;
    root.rightChild = temp;
    // Recurse on children
    mirrorTree(root.leftChild);
    mirrorTree(root.rightChild);
}
```

Exercise 11 - Solution

```
class LoggingCounter extends Counter {
    @Override
    void increment() {
        super.increment();
        System.out.println("Counter incremented");
    }
}
```

Exercise 12 - Solution

```
class PositiveFilter implements Filter<Integer> {
    @Override
    public boolean accept(Integer element) {
        return element > 0;
    }
}
```

Exercise 13 - Solution

```
static <T> ArrayList<T> filter(Filter<T> filter, ArrayList<T> list) {
    ArrayList<T> result = new ArrayList<T>();
    for (T element : list) {
        if (filter.accept(element)) {
            result.add(element);
        }
    }
}
```

```
    return result;  
}
```

Exercise 14 - Solution

```
class RangeIterator implements Iterator<Integer> {  
    private int current;  
    private int end;  
  
    RangeIterator(int start, int end) {  
        this.current = start;  
        this.end = end;  
    }  
  
    @Override  
    public boolean hasNext() {  
        return this.current < this.end;  
    }  
  
    @Override  
    public Integer next() {  
        return this.current++;  
    }  
}
```